

วิชา: CP353002 Principles of Software Design

ผู้จัดทำ: นายปวัฒน์ ปัทมมา | รหัสนักศึกษา: 673380048-6 | Section: 1

สถาปัตยกรรม: Spring Boot + JPA + PostgreSQL + Thymeleaf + Strategy Pattern

ส่วนที่ 1: Software Design & Principles Explanation

1.1 การประยุกต์ใช้ GRASP Patterns (General Responsibility Assignment Software Patterns)

ในการออกแบบระบบ Game Catalog ได้มีการจัดสรรความรับผิดชอบของคลาสตามหลักการ GRASP Patterns ดังนี้:

1. Controller Pattern:

- คลาสที่ใช้: GameController
- การทำงาน: ทำหน้าที่เป็น System Controller คอยรับเหตุการณ์ (System Events) เช่น HTTP Requests (GET, POST) จากผู้ใช้งานผ่านระบบเว็บเบราว์เซอร์ แล้วประสานงานให้ GameService ดำเนินการ และเลือกส่ง View Template (Thymeleaf) กลับไปแสดงผล

2. Information Expert:

- คลาสที่ใช้: Game (Entity Model)
- การทำงาน: คลาส Game ถือครองแอตทริบิวต์และโครงสร้างข้อมูลของเกมทั้งหมด (เช่น id, title, price, discountType) จึงเป็นผู้เชี่ยวชาญด้านข้อมูลที่เหมาะสมแก่การเก็บข้อมูลของโดเมนเกม

3. Low Coupling & High Cohesion:

- การทำงาน: แยกการทำงานออกเป็น Layered Architecture ชัดเจน (Controller → Service → Strategy / Repository) โดยแต่ละคลาสมีความรับผิดชอบเฉพาะที่เกี่ยวข้องกันสูง (High Cohesion) และไม่ยึดติดกันโดยตรง แต่องค์ประกอบต่าง ๆ ติดต่อกันผ่าน Abstraction/Interface (Low Coupling)

4. Indirection:

- **คลาสที่ใช้:** GameRepository และ DiscountContext
- **การทำงาน:** ทำหน้าที่เป็นตัวกลาง (Indirection) คั่นกลางระหว่าง Business Logic กับระบบจัดการฐานข้อมูล PostgreSQL และการคำนวณราคา ทำให้ชั้น Service ไม่จำเป็นต้องเขียนคำสั่ง SQL หรือ Logic เงื่อนไขการคำนวณราคาเอง

1.2 การประยุกต์ใช้ High-Level SOLID Principles

1. Single Responsibility Principle (SRP):

- แยกหน้าที่ชัดเจน: GameController รับผิดชอบ HTTP Route/View, GameService รับผิดชอบกฎทางธุรกิจ, DiscountStrategy รับผิดชอบการคำนวณส่วนลด, และ GameRepository รับผิดชอบการเข้าถึงฐานข้อมูล

2. Open/Closed Principle (OCP):

- ระบบเปิดต่อการขยายแต่ปิดต่อการแก้ไข หากต้องการเพิ่มโปรโมชั่นส่วนลดแบบใหม่ (เช่น ส่วนลด VIP 30%) สามารถสร้างคลาสใหม่ที่ implement DiscountStrategy ได้ทันที โดยไม่ต้องแก้ไขโค้ดเดิมใน GameService หรือ DiscountContext

3. Liskov Substitution Principle (LSP):

- ทุก Concrete Strategy (NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy) สามารถใช้งานแทน DiscountStrategy ได้อย่างสมบูรณ์โดยไม่ทำให้ระบบทำงานผิดพลาด

4. Interface Segregation Principle (ISP):

- DiscountStrategy กำหนดเมธอดเฉพาะที่จำเป็นสำหรับการคำนวณราคาเท่านั้น ไม่บังคับให้คลาสลูกต้อง implement เมธอดที่ไม่เกี่ยวข้อง

5. Dependency Inversion Principle (DIP):

- คลาสชั้นสูง (GameService, DiscountContext) พึ่งพา Abstraction (DiscountStrategy, GameRepository) ผ่าน **Constructor Injection** แทนที่จะพึ่งพา Concrete Class ตรง ๆ

1.3 การประยุกต์ใช้ Strategy Pattern ในการคำนวณส่วนลด

ระบบนี้ใช้ **Strategy Pattern** เพื่อให้อัลกอริทึมการคำนวณราคาตามประเภทส่วนลด:

- **Interface:** DiscountStrategy
- **Concrete Strategies:**
 - NoDiscountStrategy: ราคาปกติ (ส่วนลด 0%)
 - StudentDiscountStrategy: ส่วนลดนักศึกษา (ส่วนลด 10%)
 - SeasonalSaleStrategy: ส่วนลดเทศกาล (ส่วนลด 20%)
- **Context:** DiscountContext ทำหน้าที่สลับและเลือกใช้ Strategy ที่เหมาะสมตามประเภทส่วนลด โดยอัตโนมัติผ่าน Spring Dependency Injection (Map<String, DiscountStrategy>)

1.4 เหตุผลของการออกแบบ Layered Architecture

การแยก **Service Layer (GameService)** ออกจาก **Controller (GameController)** และ **Repository (GameRepository)** มีประโยชน์ดังนี้:

- **Low Coupling:** ลดความยึดติดกันระหว่างการแสดงผลบน UI กับกฎทางธุรกิจ หากมีการเปลี่ยน UI จาก Thymeleaf เป็น REST API หรือ Mobile App โค้ดใน GameService ก็ยังคงใช้งานได้ทันทีโดยไม่ต้องแก้ไข
- **High Cohesion & Reusability:** รวบรวม Business Logic เช่น การเรียกใช้ Strategy Pattern ไว้ที่เดียว ทำให้ดูแลง่ายและนำไปใช้ซ้ำได้สะดวก
- **Testability:** สามารถทำ Unit Test ชั้น Service ได้ง่ายโดยการ Mock Repository และ Strategy Context เข้าทาง Constructor

1.5 ลำดับการทำงาน (Execution Flow)

เมื่อผู้ใช้กดเพิ่มเกมใหม่จากเบราว์เซอร์:

1. **Browser → Controller:** ส่ง HTTP POST Request มาที่ /games/save พร้อมข้อมูลในรูปแบบฟอร์ม
2. **Controller → Service:** GameController รับข้อมูลแล้วส่ง Game Object ไปให้ GameService.saveGame(game)

3. **Service → Repository → DB:** GameService เรียก GameRepository.save(game) ซึ่ง Spring Data JPA จะส่งคำสั่ง SQL INSERT INTO games ... ไปบันทึกลงในฐานข้อมูล PostgreSQL
4. **Service → Strategy (ตอนดึงข้อมูลแสดงผล):** เมื่อเรียก getAllGames() ชั้น GameService จะส่ง discountType และ price ไปให้ DiscountContext คำนวณราคาสุทธิ (finalPrice) และชื่อส่วนลด (discountName) ผ่าน Strategy Pattern
5. **Controller → View (Thymeleaf):** GameController แบนรายการเกมพร้อมราคาสุทธิที่ใส่ลงใน Model แล้วส่งต่อไปยัง games/list.html แปลงเป็น HTML คืนกลับไปให้เบราว์เซอร์แสดงผล

ส่วนที่ 2: Code Implementation & Explanation

2.1 Entity Class (Game.java)

```
package com.example.demo.model;
```

```
import jakarta.persistence.*;
```

```
import java.time.LocalDate;
```

```
@Entity
```

```
@Table(name = "games")
```

```
public class Game {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    @Column(nullable = false)
```

```
    private String title;
```

@Column(nullable = false)

private String genre;

@Column(nullable = false)

private String platform;

@Column(nullable = false)

private Double rating;

@Column(nullable = false)

private LocalDate releaseDate;

@Column(nullable = false)

private Double price;

@Column(nullable = false)

private String discountType;

@Transient

private Double finalPrice;

@Transient

private String discountName;

```
// Constructors, Getters, and Setters...  
}
```

คำอธิบาย: ใช้ JPA Annotations ในการแมปคลาสกับตาราง games ใน PostgreSQL โดยฟิลด์ finalPrice และ discountName ถูกระบุเป็น @Transient เพื่อเก็บค่าคำนวณราคาสุทธิชั่วคราวผ่าน Strategy Pattern โดยไม่สร้างคอลัมน์ซ้ำซ้อนในฐานข้อมูล

2.2 Repository Interface (GameRepository.java)

```
package com.example.demo.repository;  
  
import com.example.demo.model.Game;  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
  
@Repository  
public interface GameRepository extends JpaRepository<Game, Long> {  
}
```

คำอธิบาย: Extends JpaRepository<Game, Long> ทำให้ Spring Data JPA สร้างออปเจกต์สำหรับทำ CRUD (Create, Read, Update, Delete) ให้โดยอัตโนมัติ

2.3 Strategy Pattern Implementation (strategy/)

```
// Strategy Interface  
public interface DiscountStrategy {  
    double calculateFinalPrice(double originalPrice);  
    String getDiscountName();  
}
```

```
// Concrete Strategy: Student 10%

@Component("STUDENT")

public class StudentDiscountStrategy implements DiscountStrategy {

    @Override

    public double calculateFinalPrice(double originalPrice) { return originalPrice * 0.90; }

    @Override

    public String getDiscountName() { return "ส่วนลดนักศึกษา (10%); }

}
```

```
// Strategy Context

@Component

public class DiscountContext {

    private final Map<String, DiscountStrategy> strategies;

    @Autowired

    public DiscountContext(Map<String, DiscountStrategy> strategies) {

        this.strategies = strategies;

    }

    public double calculateFinalPrice(String discountType, double originalPrice) {

        DiscountStrategy strategy = strategies.getOrDefault(discountType,

strategies.get("NONE"));

        return strategy.calculateFinalPrice(originalPrice);

    }

}
```

```
}
```

คำอธิบาย: ประยุกต์ใช้ Strategy Pattern ร่วมกับ Spring Dependency Injection โดย Spring จะฉีด Concrete Strategies เข้าสู่ Map<String, DiscountStrategy> ใน DiscountContext ตามชื่อ @Component โดยอัตโนมัติ

2.4 Service Class (GameService.java)

```
@Service
```

```
public class GameService {
```

```
    private final GameRepository gameRepository;
```

```
    private final DiscountContext discountContext;
```

```
@Autowired
```

```
public GameService(GameRepository gameRepository, DiscountContext discountContext) {
```

```
    this.gameRepository = gameRepository;
```

```
    this.discountContext = discountContext;
```

```
}
```

```
// Business Logic Methods...
```

```
}
```

คำอธิบาย: ใช้ **Constructor Injection** ในการรับ Dependencies ช่วยให้แอตทริบิวต์เป็น final เพิ่มความปลอดภัยของข้อมูล และสอดคล้องกับหลัก DIP & Low Coupling

2.5 Controller Class (GameController.java)

```
@Controller
```

```
@RequestMapping("/games")
```

```
public class GameController {
```

```
    private final GameService gameService;
```

@Autowired

```
public GameController(GameService gameService) {  
  
    this.gameService = gameService;  
  
}
```

@GetMapping({ "", "/" })

```
public String listGames(Model model) {  
  
    model.addAttribute("games", gameService.getAllGames());  
  
    return "games/list";  
  
}  
  
// Other CRUD mappings (/add, /save, /edit, /update, /delete)...  
}
```

คำอธิบาย: ทำหน้าที่ประสานงานระหว่าง HTTP Request, GameService และ Thymeleaf Views โดยใช้
วิธี **Constructor Injection** เช่นเดียวกัน

ส่วนที่ 3: Web Application & Database Screenshots

3.1 หน้าฟอร์มเพิ่มเกมใหม่ (Create Form)

The top screenshot shows the 'Game Catalog' application with a 'Add New Game' form. The form includes fields for game name, genre, platform, rating, price, strategy, and release date. The bottom screenshot shows the 'All Games' list with a table containing the details of the added game.

Game Catalog CP353002 — Lab 7

← กลับไปยังรายการเกม

+ เพิ่มเกมใหม่
กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

↓ ชื่อเกม
Roblox (673380048-6 SEC 1)

🎮 แนวเกม (Genre) 📁 แพลตฟอร์ม (Platform)
RPG / Roleplay PC

☆ คะแนน (Rating) 💰 ราคาปกติ (บาท) % ส่วนลด (Strategy)
9.0 500 ส่วนลดนักศึกษา (10%)

📅 วันวางจำหน่าย (Release Date)
02/04/2020

💾 บันทึก ❌ ยกเลิก

Game Catalog CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด
จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

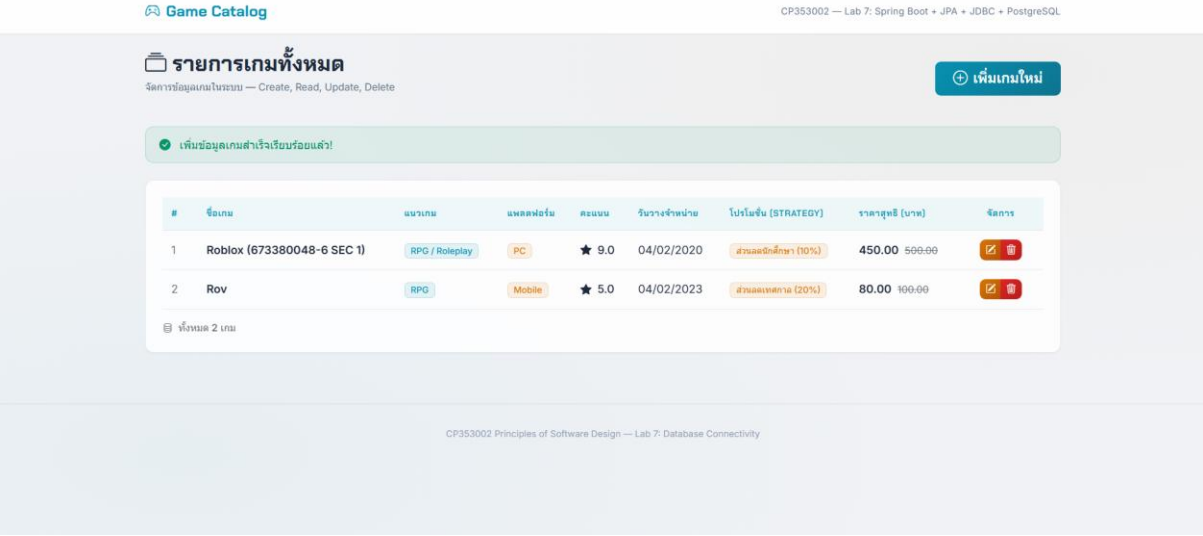
+ เพิ่มเกมใหม่

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Roblox (673380048-6 SEC 1)	RPG / Roleplay	PC	★ 9.0	04/02/2020	ส่วนลดนักศึกษา (10%)	450.00 500.00	📝 🗑️

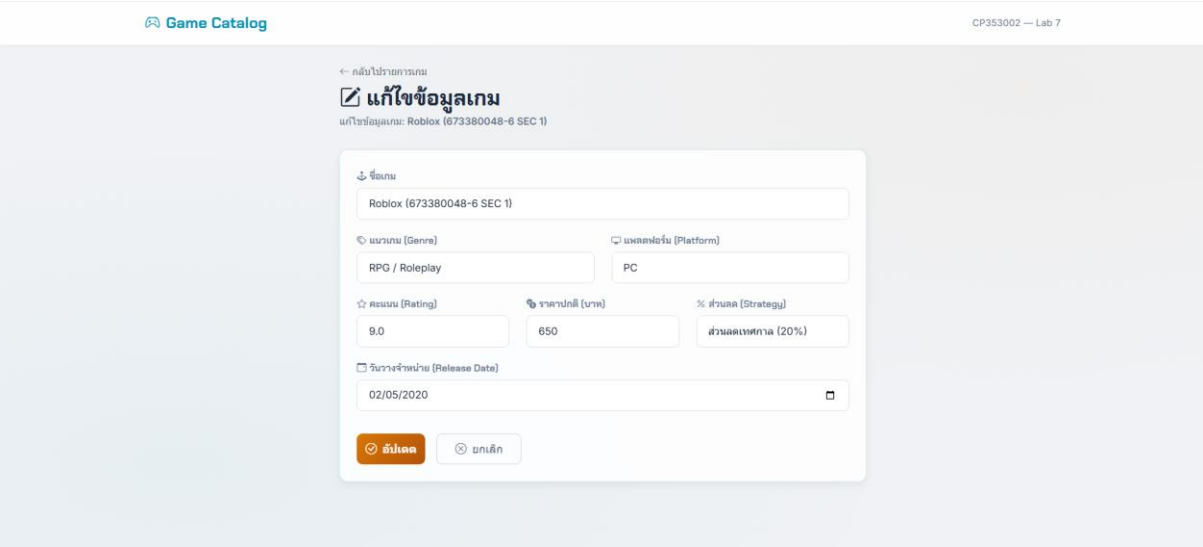
📄 ทั้งหมด 1 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

3.2 หน้าแสดงรายการเกมทั้งหมด (Read / List)



3.3 หน้าฟอร์มแก้ไขข้อมูลเกม (Update Form)



3.4 หน้ารายการเกมหลังแก้ไขสำเร็จ (Updated List)

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

+

เพิ่มเกมใหม่

✓ อัปเดตข้อมูลเกมสำเร็จเรียบร้อยแล้ว!

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชัน (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Rov	RPG	Mobile	★ 5.0	04/02/2023	ส่วนลดพิเศษ (20%)	80.00 100.00	✓ ✖
2	Roblox (673380048-6 SEC 1)	RPG / Roleplay	PC	★ 9.0	05/02/2020	ส่วนลดพิเศษ (20%)	520.00 650.00	✓ ✖

ทั้งหมด 2 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

3.5 หน้ารายการหลังลบข้อมูลเกม (Delete Success)

Game Catalog

CP353002 — Lab 7

← กลับไปรายการเกม

ยืนยันการลบ

!

คุณแน่ใจหรือไม่?

การดำเนินการนี้จะลบข้อมูลเกมออกจากระบบอย่างถาวร

ชื่อเกม

Rov

แนวเกม

RPG

แพลตฟอร์ม

Mobile

คะแนน

★ 5.0

ราคา

100.00 บาท

ยืนยันลบ

← ยกเลิก

Game Catalog

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

+

เพิ่มเกมใหม่

✓ ลบข้อมูลเกมออกจากระบบเรียบร้อยแล้ว!

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชัน (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Roblox (673380048-6 SEC 1)	RPG / Roleplay	PC	★ 9.0	05/02/2020	ส่วนลดพิเศษ (20%)	520.00 650.00	✓ ✖

ทั้งหมด 1 เกม

CP353002 Principles of Software Design — Lab 7: Database Connectivity

3.6 หน้าจอตรวจสอบข้อมูลใน PostgreSQL Database

The screenshot displays the pgAdmin 4 web interface. On the left, the 'Object Explorer' shows a tree structure of database objects, with 'public' and 'games' tables selected. The main pane shows a SQL query: `SELECT * FROM public.games ORDER BY id ASC`. Below the query, the 'Data Output' tab displays a single row of data from the 'games' table. The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.135'.

id	discount_type	genre	platform	price	rating	release_date	title
1	SEASONAL	RPG / Roleplay	PC	650	9	2020-02-05	Roblox (673380048-6 SEC...